```
var_dump($res);
```

```
Output:
  ...
  ["HTTP_USER_AGENT"]=>
  string(3) "PHP"
  ["HTTP_X_FOO"]=>
  string(8) "foo, bar"
  ...
```

You'll need to run this via a CLI PHP binary that isn't containerized so it is able to access port 8080 of the host, or make sure your CLI container is able to communicate with your server container (a fun exercise for the reader). Notice that the `HTTP_X_FOO` value is assigned `foo, bar`, which is the semantics for a value list in the HTTP RFC. If you find yourself doing any heavy lifting with making HTTP requests in PHP, I highly recommend the Guzzle library (which has a fairly painless interface for making promise-based asynchronous requests to speed things up), which you can find at https://github.com/guzzle/guzzle.

Cookies

Closely related to headers are cookies. These are the things that recent EU law required websites to warn us about on the first visit, generally with annoying pop-ups. You'll probably know that they're little snippets of data that usually wind up clogging up some corner of your browser profile data (e.g., Chrome implements persistent cookies as an on-disk SQLite database). Cookies are simple at heart; they're just text-based key/value pairs. They are passed to the server via the `Cookie:` request header (and can be set by the server in the `Set-Cookie:` response header); each cookie pair in the header is separated by a semicolon. When cookies are set, it is usually with some metadata to bind them to a domain (or sub-domain) and path with an expiry timestamp (any cookies set without an expiry will be treated as "session-only" and vanish once the browser is closed). On each request the browser will check its cookie storage and find all those that match the domain and path URL components and haven't expired yet.